SASD GmbH

downloads.sasd.de

Installations- und Setup-Leitfaden

Version 0.1.0

Dokumenttyp	Stand	Zielsystem
Setup- und Installationsleitfaden	2026-03-31	PHP 8.2, dateibasierter Katalog
Sprache	**Projekt**	**Zielgruppe**
Deutsch	downloads-sasd	Administrator / Betreiber

Zweck des Dokuments

Dieser Leitfaden beschreibt die technische Erstinstallation, Inbetriebnahme, Grundkonfiguration und den operativen Schnellstart der ersten downloads-sasd-Version. Er ergänzt das Admin-Handbuch und konzentriert sich bewusst auf den praktisch ausführbaren Setup-Pfad.

1. Dokumentüberblick

Das Projekt downloads-sasd ist eine leichtgewichtige PHP-Anwendung für das zukünftige Portal downloads.sasd.de. Die erste Version ist bewusst überschaubar aufgebaut: HTML-Ausgabe über PHP-Views, Routing über einen Front Controller, fachliche Logik in Services und Zugriff auf Katalogdaten über Repository-Schnittstellen.

Die Anwendung nutzt aktuell JSON-Dateien als Katalogquelle. Dadurch bleibt der operative Aufwand niedrig, während die Architektur bereits so vorbereitet ist, dass eine spätere Datenbankmigration vor allem in der Repository- und Storage-Schicht stattfindet.

Abgrenzung
Dieses Dokument beschreibt die technische Installation und Inbetriebnahme. Für Betrieb, Pflegeprozesse, Katalogorganisation und konzeptionelle Entscheidungen ist das Admin-Handbuch die führende Referenz.

1.1 Zielgruppe

Der Leitfaden richtet sich an die Person, die das System lokal einrichtet, auf einem Server bereitstellt oder ein späteres Deployment vorbereitet. Im aktuellen Projektstand ist dies in erster Linie der Betreiber selbst.

1.2 Lieferumfang der ersten Version

- öffentliche Startseite mit Such- und Katalogeinstieg
- Produkt- und Artefaktseiten
- Mehrsprachigkeit mit Deutsch und Englisch
- dateibasierter Katalog unter data/catalog/
- statische Artefaktablage unter storage/files/
- CLI-Skripte für Validierung und Katalog-Rebuild

2. Technischer Überblick

Die Projektstruktur trennt öffentliche Einstiegspunkte, Anwendungscode, Konfiguration, Ansichtslogik, Katalogdaten und ausgelieferte Dateien. Diese Trennung ist für Setup und späteren Betrieb wichtig, weil Änderungen an Inhalten, Applikationslogik und Deployment dadurch sauber voneinander getrennt bleiben.

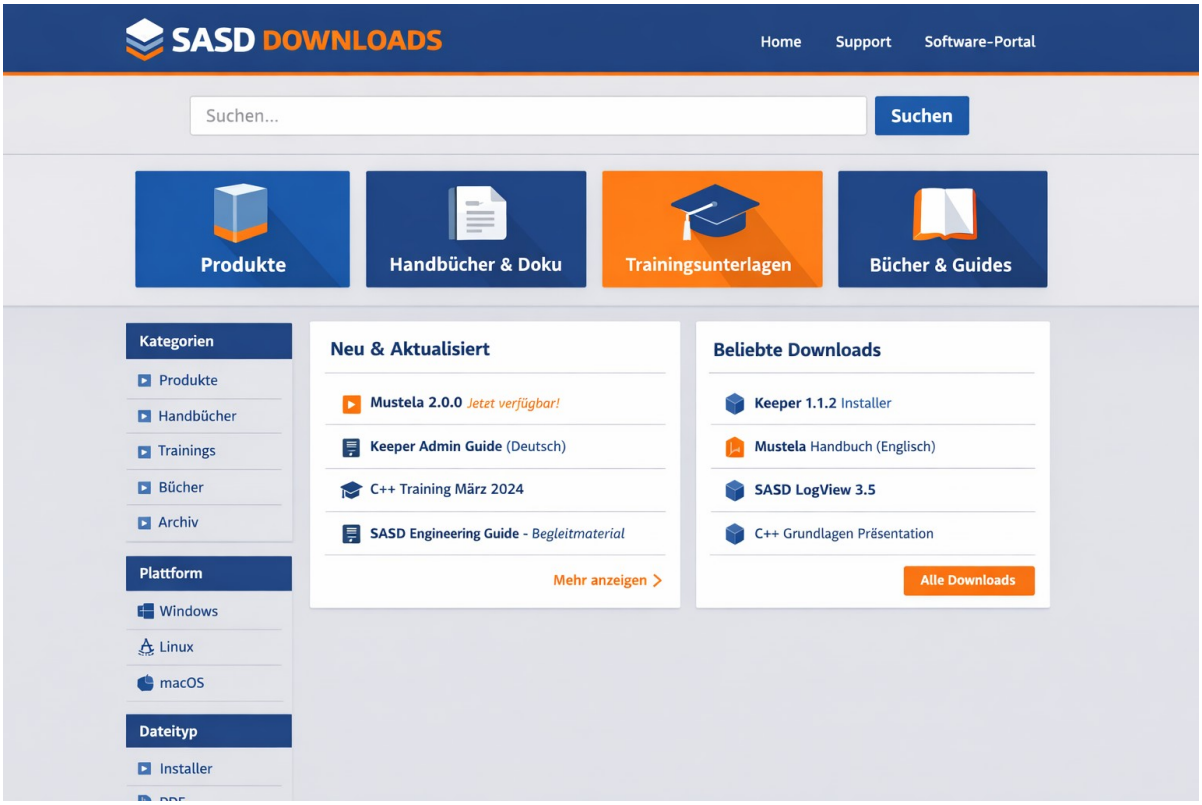

Abbildung 1 - Mockup der Startseite von downloads.sasd.de

2.1 Relevante Verzeichnisse

Pfad	Rolle	Bemerkung
public/	öffentlicher Webroot	enthält index.php und Assets
app/	Bootstrap / Wiring	verdrahtet Konfiguration und Services
src/	Applikationscode	Controller, Services, Repositories, Entities
resources/lang/	Sprachdateien	UI-Texte für de und en
resources/views/	Templates	HTML-Ausgabe der Anwendung
config/	Konfiguration	zentrale Anwendungseinstellungen
data/catalog/source/	Quellkatalog	editierbare Produkt- und Artefaktmetadaten
data/catalog/generated/	generierter Katalog	wird per CLI neu erzeugt
storage/files/	Artefakte	ZIP, PDF und weitere Download-Dateien
cli/	Wartungsskripte	validate und rebuild

3. Systemvoraussetzungen

Für die erste Version genügt eine schlanke PHP-Laufzeitumgebung. Da die Anwendung derzeit keine Datenbank und keine komplexen Hintergrunddienste benötigt, bleiben Installation und Hosting bewusst einfach.

3.1 Mindestvoraussetzungen

Komponente	Anforderung
PHP	Version 8.2 oder neuer
Webserver	für lokale Entwicklung optional nur PHP Built-in Server, für später Apache oder Nginx
Dateisystemzugriff	Leserechte auf Projektdateien und Artefaktablage
CLI-Zugriff	für php cli/validate-catalog.php und php cli/rebuild-catalog.php
Betriebssystem	Linux bevorzugt, lokal auch Windows oder macOS möglich

Hinweis zur ersten Version
Composer, Datenbankserver, Queue-Systeme oder ein separates Backend-Framework sind für die Inbetriebnahme von v0.1.0 nicht erforderlich. Das ist bewusst so gewählt, um den Setup-Pfad schlank zu halten.

4. Lokale Installation und erster Start

Der schnellste Weg zum Einstieg ist eine lokale Ausführung mit dem eingebauten PHP-Webserver. Diese Methode eignet sich für Architekturtests, Layout-Prüfung, erste Katalogpflege und die inhaltliche Weiterentwicklung.

4.1 Projekt bereitstellen

- Projekt aus dem Git-Repository klonen oder den bereitgestellten Projektordner lokal entpacken.
- Sicherstellen, dass die Verzeichnisstruktur unverändert erhalten bleibt.
- Für lokale Tests das Projektwurzelverzeichnis als Arbeitsverzeichnis verwenden.

4.2 Integrierten Webserver starten

```
cd /pfad/zum/downloads-sasd
php -S 127.0.0.1:8080 -t public
```

Danach ist die Anwendung unter http://127.0.0.1:8080 erreichbar.

4.3 Funktionstest

- Startseite aufrufen und prüfen, ob Navigation, Suche und Kartenansicht geladen werden.
- Produktseite und Artefaktseite aufrufen.
- Einen Download-Link testen.
- Katalog-Validierung und Rebuild einmal manuell ausführen.

5. Katalogprüfung und Katalogaufbau

Die Katalogdateien unter data/catalog/source/ sind die editierbare Quelle. Die Anwendung greift im regulären Betrieb auf den generierten Katalog unter data/catalog/generated/catalog.json zu. Nach strukturellen Änderungen an Produkten oder Artefakten sollte daher immer validiert und anschließend neu aufgebaut werden.

5.1 Validierung

```
php cli/validate-catalog.php
```

Die Validierung dient dazu, inkonsistente Metadaten früh zu erkennen. Sie sollte vor jedem produktiven Katalog-Rebuild und besonders vor einem Deployment ausgeführt werden.

5.2 Rebuild

```
php cli/rebuild-catalog.php
```

Der Rebuild erzeugt den zusammengeführten Katalog, der von der Anwendung gelesen wird. Nach Änderungen an products.json, artifacts.json oder am Status von Einträgen ist dieser Schritt obligatorisch.

Empfohlene Reihenfolge
1. Quelldaten anpassen. 2. Dateien im Storage ablegen. 3. Validierung starten. 4. Rebuild ausführen. 5. Funktionsprüfung im Browser durchführen.

6. Deployment auf einen Server

Für einen echten Serverbetrieb sollte der Webserver ausschließlich auf public/ zeigen. Alle übrigen Verzeichnisse bleiben außerhalb des öffentlichen Webroots erreichbar und sind damit nicht direkt per URL abrufbar.

6.1 Webroot

Der korrekte Webroot ist das Verzeichnis public/. Dadurch bleibt die Anwendungsstruktur geschützt, und nur der Front Controller sowie die vorgesehenen Assets werden öffentlich ausgeliefert.

6.2 Beispielhafte Zielstruktur

```
/var/www/downloads-sasd/
├── app/
├── cli/
├── config/
├── data/
├── public/
├── resources/
├── src/
└── storage/
```

6.3 Dateirechte

- Webserver-Prozess benötigt Leserechte auf Anwendungscode, Sprachdateien, Views, Konfiguration und generated catalog.
- Artefaktdateien unter storage/files/ müssen lesbar sein.
- Schreibrechte werden in der ersten Version im Regelfall nur für den Betreiber während Pflege- oder Build-Schritten benötigt.

6.4 Deployment-Ablauf

- Neue Projektversion in ein Staging-Verzeichnis ausrollen.
- Katalog validieren und rebuilden.
- Smoke-Test gegen die neue Instanz durchführen.
- Webroot oder Symlink auf die neue Version umschalten.

7. Inhalte anlegen, ändern und ausliefern

Die erste Version ist für einen dateibasierten Pflegeprozess optimiert. Inhalte werden nicht über ein Browser-Backend erstellt, sondern über strukturierte Dateien und Metadaten im Repository oder auf dem Server gepflegt.

7.1 Neues Produkt aufnehmen

- Produktmetadaten in data/catalog/source/products.json ergänzen.
- Zugehörige Artefakte in storage/files/ im passenden Pfad ablegen.
- Artefakte in data/catalog/source/artifacts.json beschreiben.
- Validierung und Rebuild ausführen.
- Produktseite und Downloads testen.

7.2 Statuswerte richtig einsetzen

Die Statuswerte current, lts, deprecated, archived und hidden helfen dabei, Inhalte nicht vorschnell zu löschen. Für einen professionellen Betrieb ist diese Unterscheidung wichtig, weil Benutzer so kontrolliert auf aktuelle, ältere oder absichtlich verborgene Artefakte geführt werden können.

Status	Verwendung
current	aktuell sichtbare Standardversion
lts	langfristig gepflegte oder bevorzugte Version
deprecated	noch sichtbar, aber veraltet
archived	historisch weiterhin verfügbar
hidden	technisch vorhanden, aber nicht regulär sichtbar

7.3 Dateien physisch entfernen

Physisches Löschen sollte erst dann erfolgen, wenn eindeutig feststeht, dass ein Artefakt nicht mehr benötigt wird. In frühen Produktphasen ist es meist besser, die Sichtbarkeit zunächst über hidden oder archived zu steuern.

8. Mehrsprachigkeit von Anfang an richtig vorbereiten

Die erste Version unterstützt Deutsch und Englisch bereits produktiv. Die Struktur ist zugleich so vorbereitet, dass weitere Sprachen später ergänzt werden können, ohne Routing, Views oder die grundsätzliche Architektur neu zu entwerfen.

8.1 Wo die Sprachinformationen liegen

- resources/lang/de.php und resources/lang/en.php enthalten UI-Texte.
- Lokalisierte Inhalte für Produkte und Artefakte liegen im Katalog.
- Die Sprachwahl wird zentral über die Localization-Komponenten aufgelöst.

8.2 Erweiterung um weitere Sprachen

- neue Sprachdatei in resources/lang/ anlegen, zum Beispiel fr.php oder es.php
- Locale in config/app.php aktivieren
- produkt- und artefaktbezogene Inhaltstexte um den neuen Sprachschlüssel ergänzen
- Layout und Zeichensatz bei rechts-nach-links-Sprachen sowie ostasiatischen Schriften gesondert prüfen

Sprachstrategie
Für eine spätere Internationalisierung sollte zwischen UI-Übersetzungen und fachlichen Kataloginhalten sauber getrennt bleiben. Genau diese Trennung ist in v0.1.0 bereits angelegt.

9. Sicherheit, Backup und Betriebshygiene

Obwohl die Anwendung in der ersten Version bewusst einfach gehalten ist, sollte der Betrieb nicht improvisiert werden. Gerade bei einem Download-Portal ist wichtig, dass nur die vorgesehenen Dateien ausgeliefert werden, dass Katalogänderungen nachvollziehbar bleiben und dass ein Wiederherstellungspfad existiert.

9.1 Mindestmaßnahmen

- Webroot ausschließlich auf public/ setzen.
- Quellverzeichnisse nicht direkt öffentlich exponieren.
- Deployment nur aus geprüften Projektständen durchführen.
- Vor Änderungen an Katalog oder Artefaktbestand Sicherung anlegen.
- Download-Dateien nur kontrolliert unter storage/files/ ablegen.

9.2 Backup-Empfehlung

Für v0.1.0 reicht in der Regel ein dateibasiertes Backup von Projektcode, Quellkatalog, generiertem Katalog und storage/files/. Für produktive Nutzung sollten mindestens tägliche Sicherungen und eine getestete Wiederherstellung vorgesehen werden.

10. Fehlerbilder und schnelle Lösungen

Symptom	Prüfung / Lösung
Startseite lädt nicht	Prüfen, ob der Webserver wirklich auf public/ zeigt oder lokal mit -t public gestartet wurde.
Produkte erscheinen nicht	Quelldaten prüfen, danach Validierung und Rebuild ausführen.
Download-Link führt ins Leere	Pfad in artifacts.json mit storage/files/ abgleichen und Dateinamen exakt prüfen.
Neue Sprache erscheint nicht	Sprachdatei anlegen, Locale aktivieren und Inhalte im Katalog ergänzen.

Änderungen im Browser nicht sichtbar	Generated catalog neu aufbauen und Browser-Cache bzw. Server-Cache ausschließen.

11. Schnellreferenz für den Betreiber

11.1 Standardbefehle

```
php -S 127.0.0.1:8080 -t public
php cli/validate-catalog.php
php cli/rebuild-catalog.php
```

11.2 Empfohlene Erstcheckliste

- Projekt vollständig entpackt oder geklont
- PHP 8.2 verfügbar
- Webroot korrekt gesetzt
- Startseite erreichbar
- Validierung erfolgreich
- Rebuild erfolgreich
- mindestens ein Download getestet